
Data Structures

BS (CS) _Fall_2025

Lab_15 Manual

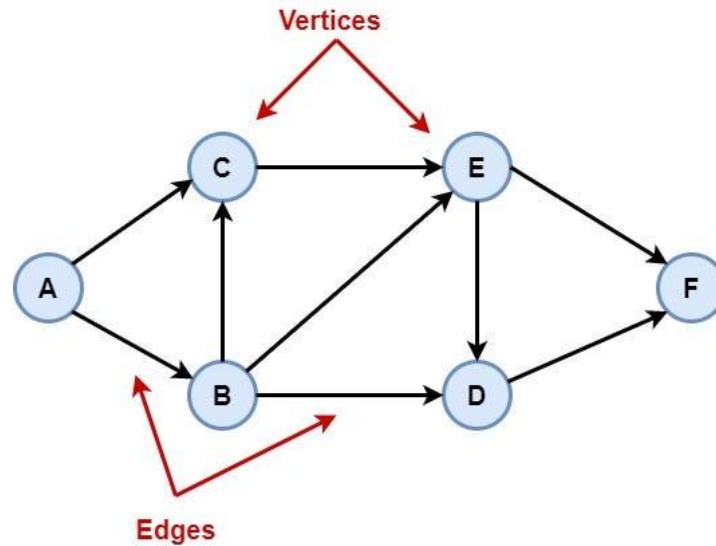


Learning Objectives:

1. Graphs using Adjacency List

Graph Data Structure

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by $G(V, E)$.



Representations of Graph

Here are the two most common ways to represent a graph: For simplicity, we are going to consider only **unweighted graphs** in this post.

1. Adjacency Matrix
2. Adjacency List

Adjacency Matrix Representation

Covered in the previous lab.

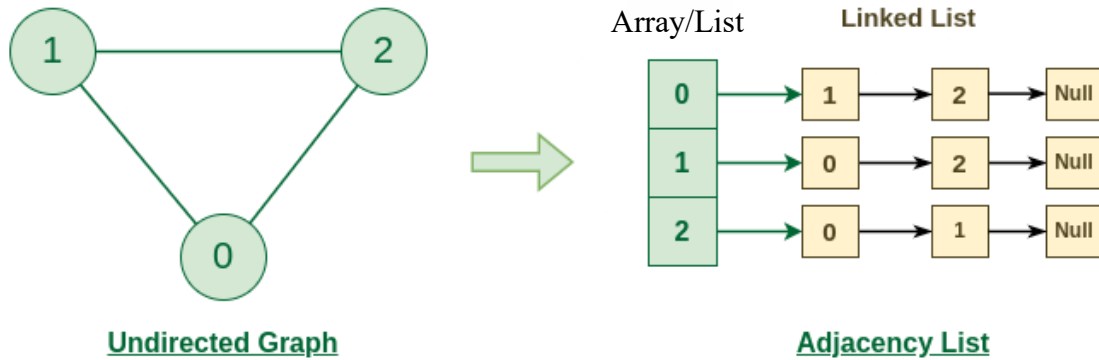
Adjacency List

An adjacency list represents a graph as an array/list of linked lists.

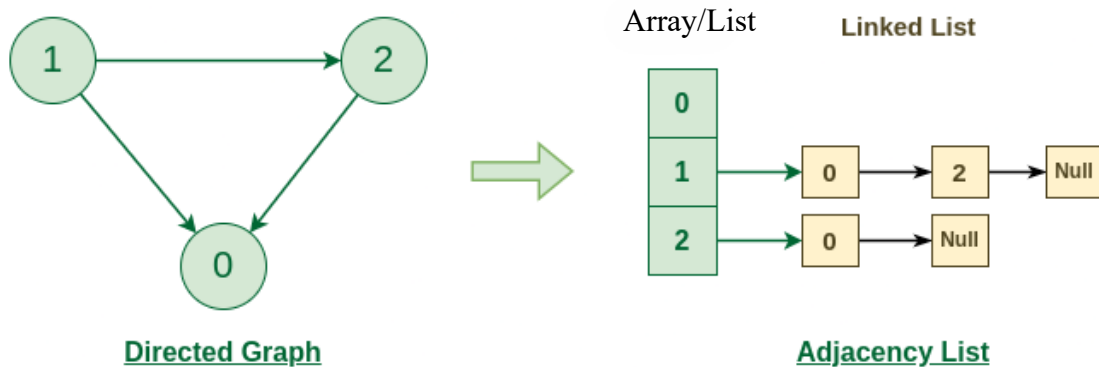
The index of the array/list represents a vertex and each element in its linked list represents the other vertices that form an edge with the vertex.

Representation of Undirected Graph as Adjacency List:

For undirected Graph



For directed Graph



Pros of Adjacency List

An adjacency list is efficient in terms of storage because we only need to store the values for the edges. For a graph with millions of vertices, this can mean a lot of saved space.

Cons of Adjacency List

Finding the adjacent list is not quicker than the adjacency matrix because all the connected nodes must be first explored to find them.

Graph Traversal

Breadth first search

Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

Algorithm

A standard BFS implementation puts each vertex of the graph into one of two categories:

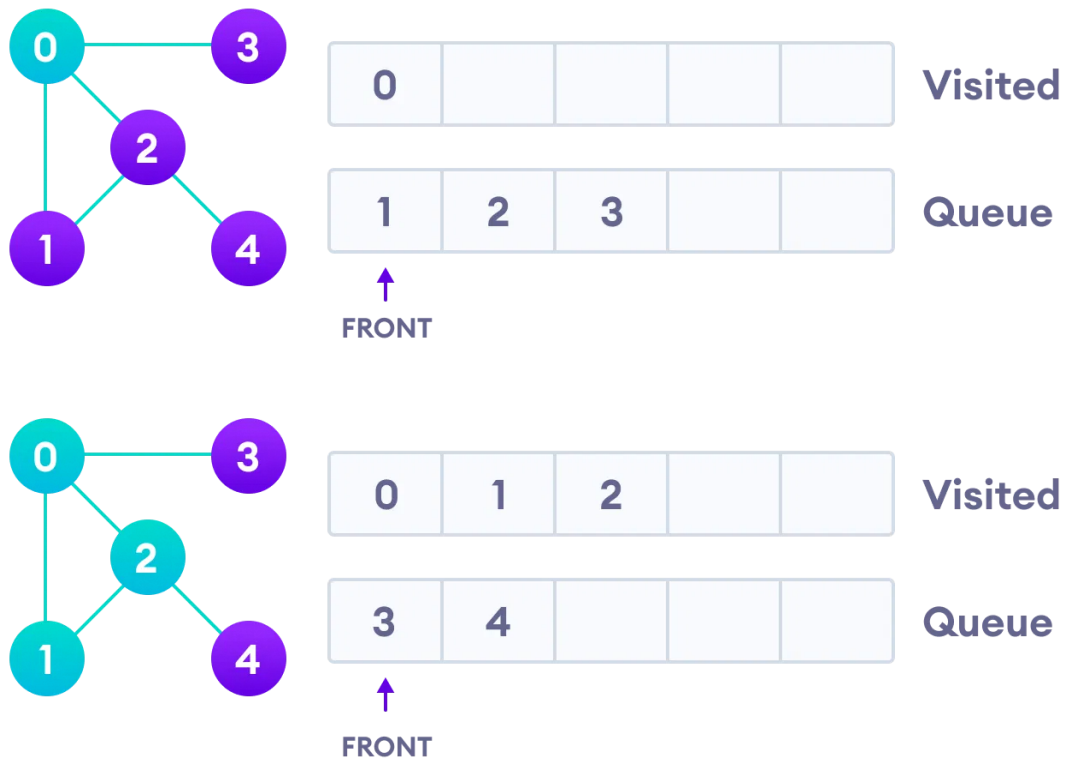
1. Visited
2. Not Visited

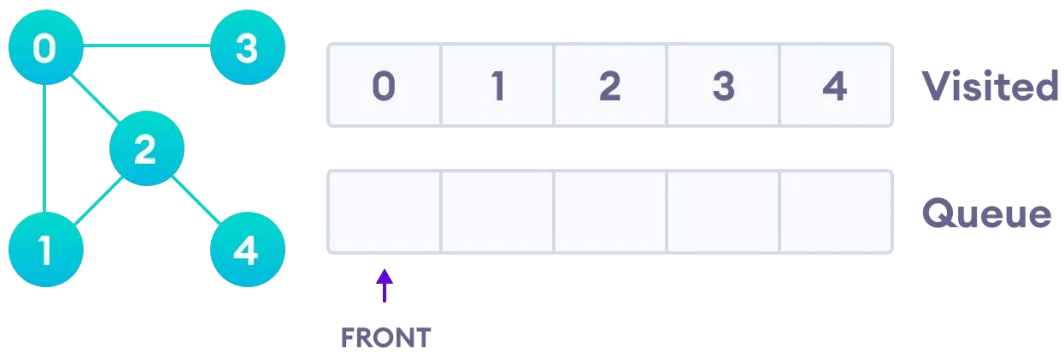
The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The algorithm works as follows:

1. Start by putting any one of the graph's vertices at the back of a queue.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

The graph might have two different disconnected parts so to make sure that we cover every vertex, we can also run the BFS algorithm on every node





Depth first search

Depth First Search (DFS) is a graph traversal method that starts from a source vertex and explores each path completely before backtracking and exploring other paths. To avoid revisiting nodes in graphs with cycles, a visited array is used to track visited vertices.

Algorithm

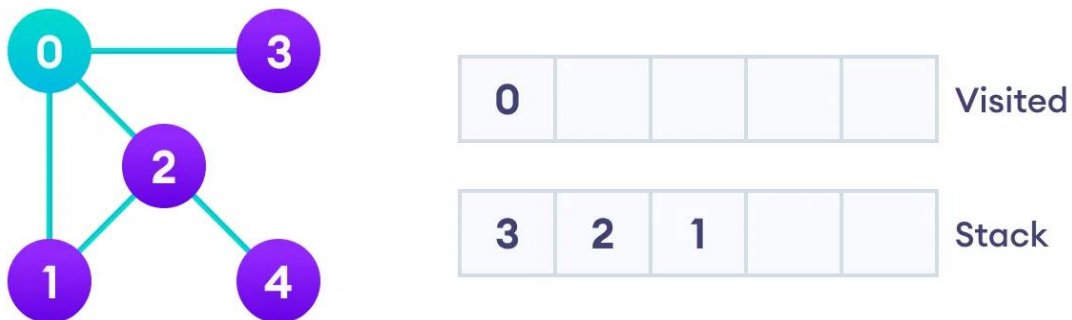
A standard DFS implementation puts each vertex of the graph into one of two categories:

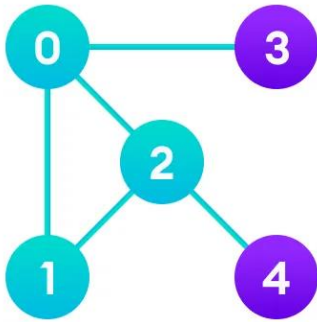
1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The DFS algorithm works as follows:

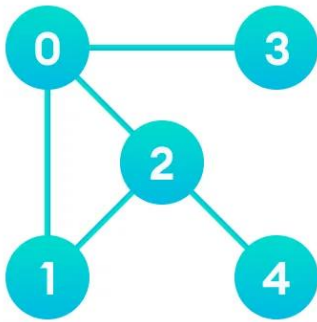
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.





0	1	2			Visited
---	---	---	--	--	---------

3	4				Stack
---	---	--	--	--	-------



0	1	2	4	3	Visited
---	---	---	---	---	---------

					Stack
--	--	--	--	--	-------